13-Stage Pre-Deployment Security Gate

Implementation and verification guide

Serveur Railway – Node.js / Express / Railway

Pipeline file:	`.github/workflows/security.yml`
Deployment target:	Railway
Guide date:	10 September 2026

Contents

1 What was implemented

Every push to `main`, every pull request, the weekly schedule, and every manual run starts one GitHub Actions job named **All 13 DevSecOps stages**. Component steps use `continue-on-error` so one failure does not hide the results of later independent tests. The final **Evaluate final deployment gate** step reads every outcome and fails the whole job if any required stage did not pass.

The pipeline builds the container once, scans that candidate, generates its SBOM, attests it, signs it, and runs ZAP against that same image. Pull requests run all testable checks; SLSA provenance and keyless signing run only for release-capable, non-PR executions because they require GitHub OIDC identity.

1.1 Files added or changed

File	Purpose
`.github/workflows/security.yml`	Orchestrates and enforces all thirteen stages.
`Dockerfile`, `.dockerignore`	Reproducible non-root candidate used by CI and Railway.
`railway.json`	Selects the Dockerfile and configures `/health`.
`.security/semgrep/rules.yml`	Project SAST rules.
`.security/license-policy.json`	Approved and denied license identifiers.
`.security/security-policy.json`	Machine-readable repository and container requirements.
`scripts/check-*.js`	Syntax, license, and policy evaluators.
`scripts/verify-security-gate.js`	Final fail-closed aggregation and summary.
`test/security.integration.test.js`	Six HTTP security integration tests.

2 How to tell whether deployment is allowed

1. Open the repository on GitHub and select **Actions** → **Pre-deployment security gate**.
2. Open the run for the exact commit you intend to deploy.
3. Confirm **All 13 DevSecOps stages** is green and its final step says `SECURITY GATE PASSED`.
4. Read the run summary table: each numbered row must say `success`. On a pull request only, stages 9 and 10 may say `skipped`; they will run on the subsequent push to `main`.
5. Download `security-reports-<commit SHA>` from the run's **Artifacts** section when detailed evidence is needed. The authoritative machine result is `gate-summary.json`.

Important: a green workflow blocks Railway only after **Wait for CI** is enabled for the production service. In Railway, open the service, go to its deployment/source settings, enable **Wait for CI**, and ensure the connected deployment branch is `main`. Railway then holds the deployment in `WAITING` until this workflow finishes. See Railway: Controlling GitHub Autodeploys.

3 1. Secrets detection

Purpose. TruffleHog examines the complete Git history, not only the current files, for credentials that can be verified against their provider. This catches a token even if a later commit deleted it.

Pass rule. The scanner exits zero and finds no verified secret. **Evidence:** `secrets.txt` plus the stage log.

How to verify. Expand **1 - Secrets detection (TruffleHog)** and confirm the step is green. If it fails, revoke and rotate the credential first, remove it from current files, and clean Git history where appropriate. Merely deleting the latest copy is insufficient.

4 2. Code quality

Purpose. Node checks every project JavaScript file for parse errors, then ESLint enforces correctness and dangerous-language rules such as no `eval`, no implied evaluation, unreachable-code checks, and invalid control flow.

Pass rule. Both `npm run syntax` and `npm run lint` exit zero. **Evidence:** `code-quality.txt`.

Local verification.

```
npm ci
npm run check
```

The last command must exit with code 0 and show no ESLint errors.

5 3. SAST – static application security testing

Purpose. Semgrep analyzes source structure without running the application. The checked-in rules reject dynamic evaluation, shell-based child-process execution, weak MD5/SHA-1 password hashing, and a known default administrator password.

Pass rule. No rule with severity `ERROR` matches. **Evidence:** `sast.json`.

How to verify. A passing report has an empty `results` array and the step exits zero. A finding contains a rule ID, path, and line number; fix the data flow or unsafe API rather than suppressing the rule without review.

6 4. SCA – dependency vulnerability analysis

Purpose. Two independent databases inspect the locked dependency tree: npm Advisory Database through `npm audit`, and OSV through OSV-Scanner. Production high/critical vulnerabilities block release.

Pass rule. Both scanners exit zero. **Evidence:** `npm-audit.json` and `osv.txt`.

Local verification.

```
npm ci
npm audit --omit=dev --audit-level=high
```

At implementation time this command reports zero known vulnerabilities. A future advisory can correctly turn a previously green commit red; update or replace the affected dependency and rerun the gate.

7 5. Infrastructure as code scan

Purpose. Trivy's configuration scanner checks the Dockerfile, workflow, and supported configuration for high/critical deployment misconfiguration.

Pass rule. No high or critical IaC finding. **Evidence:** `iac.json`.

How to verify. Confirm the report contains no failed result at the configured threshold. Typical fixes include running as a non-root user, narrowing permissions, adding a health check, or removing credentials from build arguments.

8 6. License compliance

Purpose. The checker inventories every locked npm package and compares each SPDX expression with the checked-in allow/deny policy. Missing, denied, and unreviewed identifiers fail closed.

Pass rule. Every dependency license is classified and approved. **Evidence:** `licenses.json` and `license-check.txt`.

Local verification.

```
npm run security:licenses
```

GPL/LGPL occurrences used by media tooling are explicitly documented for legal follow-up. Approval in this technical policy is not a substitute for legal advice, especially when distributing the container.

9 7. Container scan

Purpose. Trivy scans the built image, including Debian packages and production npm packages, rather than assuming the source lockfile describes everything shipped.

Pass rule. No fixable high or critical vulnerability is present. **Evidence:** `container-scan.json`.

How to verify. The scan must target `serveur-railway:<commit SHA>` and exit zero. Fix findings by updating the base image or the affected package, rebuild, and scan the new image.

10 8. SBOM generation

Purpose. Syft creates a CycloneDX JSON Software Bill of Materials from the saved candidate image. It records the components needed for incident response, vulnerability matching, and customer evidence.

Pass rule. Syft exits zero and produces valid, non-empty JSON. **Evidence:** `sbom.cdx.json`.

How to verify. Open the file and confirm `bomFormat` is `CycloneDX`, `components` is populated, and the run's stage is green. Generation success means the inventory exists; it does not mean all components are vulnerability-free–stage 7 provides that decision.

11 9. SLSA provenance

Purpose. `actions/attest` binds the candidate image archive's SHA-256 digest to GitHub's workflow identity using an in-toto/SLSA provenance statement. This answers who built which bytes, from which workflow and commit.

Pass rule. GitHub accepts and signs the attestation for the archive. The evidence appears under the repository's attestations and in the step output.

How to verify. Download the image archive from a trusted release store if retained, then use GitHub CLI:

```
gh attestation verify application-image.tar --repo OWNER/REPOSITORY
```

The verified subject digest must match the archive and the repository/workflow identity must be expected. GitHub documents this at actions/attest.

12 10. Image signing

Purpose. Cosign uses GitHub's short-lived OIDC identity to sign the candidate container archive as a blob. No long-lived private signing key is stored in repository secrets. The workflow immediately verifies its own bundle.

Pass rule. Both `cosign sign-blob` and `cosign verify-blob` exit zero. **Evidence:** `image.sigstore.json`.

Independent verification.

```
cosign verify-blob --bundle image.sigstore.json \
  --certificate-identity \
  https://github.com/OWNER/REPO/.github/workflows/security.yml@refs/heads/main \
  --certificate-oidc-issuer https://token.actions.githubusercontent.com \
  application-image.tar
```

Verification must report success and the expected identity. A valid signature proves integrity and signer identity; stages 3, 4, 5, 7, 11, 12, and 13 determine security fitness.

13 11. Policy as code

Purpose. Repository rules are data in `.security/security-policy.json` and are evaluated automatically. Current requirements include mandatory security files, a non-root final container user, no `:latest`

base image, a Docker health check, excluded secrets/runtime data, a `/health` route, and no default admin password.

Pass rule. Every declared policy assertion evaluates true. **Evidence:** `policy.json` and `policy-check.txt`.

Local verification.

```
npm run security:policy
```

Changes to policy should be reviewed like application code because they alter what can be deployed.

14 12. DAST – OWASP ZAP

Purpose. CI starts the actual candidate image with WhatsApp networking disabled and safe test credentials, waits for `/health`, then OWASP ZAP crawls and passively scans the HTTP surface over an isolated Docker network.

Pass rule. The application becomes healthy and ZAP has no blocking baseline failure. Informational and warning observations remain in the report for review. **Evidence:** `zap.html` and `zap.json`.

How to verify. Download and open `zap.html`. Confirm the target is `http://security-app:8080`, inspect every alert, and confirm the CI stage is green. The checked-in `zap-rules.tsv` promotes missing cache, anti-clickjacking, MIME-sniffing, HSTS, CSP, and server-disclosure controls to `FAIL`. The `-I` setting prevents other warning-only baseline observations from blocking; technical scan errors and promoted controls still block.

15 13. Integration tests

Purpose. Node starts the real Express module in a child process with temporary data and verifies behavior across component boundaries.

The suite currently proves:

- `/health` is public and returns a healthy response;
- CSP, clickjacking, MIME-sniffing, and permissions headers are present;
- unapproved `Host` headers are rejected;
- protected APIs reject anonymous requests;
- login rejects a missing CSRF token;
- login remains available when only the two web credential variables are configured; and
- form bodies over 10 KB are rejected.

Pass rule. All tests pass with zero failures. **Evidence:** `integration-tests.txt`.

Local verification.

```
npm test
```

The expected result is `tests 7`, `pass 7`, `fail 0`.

16 Required one-time Railway and GitHub setup

1. In Railway, set only `WEB_ADMIN_USER` and `WEB_ADMIN_PASSWORD` as required service variables. Do not commit them. Server name, role, allowed hosts, and WhatsApp use their built-in defaults. Turnstile is optional and is enabled only when both Turnstile keys are configured. If `auth.json` already exists on a persistent volume, plan credential rotation separately.
2. In the Railway service connected to `main`, enable **Wait for CI**. Without this dashboard switch, GitHub can report failure but Railway may already have started an automatic deployment.
3. Protect `main` in GitHub and require the status check **All 13 DevSecOps stages** before merge. Prevent administrators from bypassing it unless emergency policy explicitly permits that.

4. Ensure GitHub artifact attestations are available for the repository plan. Public repositories are supported broadly; private/internal availability can depend on the GitHub plan. A missing capability intentionally makes stage 9 fail closed.
5. Keep Docker/action versions maintained. Dependabot or an equivalent update process should propose reviewed upgrades; never blindly change a security scanner to an unreviewed mutable release.

17 Daily operator checklist

☐	The green workflow belongs to the exact commit being deployed.
☐	The summary says `SECURITY GATE PASSED`.
☐	All 13 release-stage outcomes say `success`.
☐	Railway **Wait for CI** is enabled and the deployment left `WAITING` only after success.
☐	Unexpected ZAP warnings and copyleft license entries were reviewed.
☐	The SBOM, scan JSON, signature bundle, and GitHub attestation are retained with the release evidence.

18 What a red gate means

A red gate is a deployment decision, not merely a report. Open the first failed numbered stage, use its detailed artifact to locate the cause, remediate it, and push a new commit. Do not rerun until green to hide intermittent failures. If a scanner service is unavailable, the pipeline fails closed; record and review any emergency override outside the pipeline.

19 Reference links

- Railway Wait for CI: https://docs.railway.com/deployments/github-autodeploys
- GitHub artifact attestations action: https://github.com/actions/attest
- Semgrep: https://github.com/semgrep/semgrep
- OSV-Scanner: https://github.com/google/osv-scanner
- Trivy: https://github.com/aquasecurity/trivy
- Syft: https://github.com/anchore/syft
- Cosign: https://github.com/sigstore/cosign
- OWASP ZAP baseline scan: https://www.zaproxy.org/docs/docker/baseline-scan/
- CycloneDX: https://cyclonedx.org/
- SLSA: https://slsa.dev/